

Recapitulação de Python e POO

DO CÓDIGO ISOLADO

ao RPG funcional

Thoric, Elara, Goblin e Orc voltam ao terminal.

Agora cada método testado vira parte de uma batalha completa.

MISSÃO DA AULA

```
$ python main.py
```

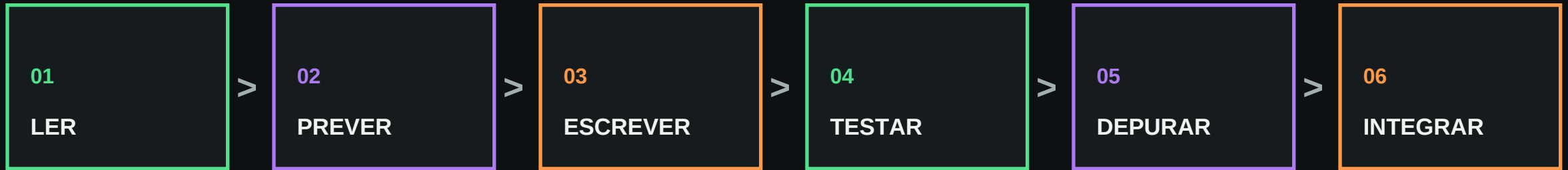
```
[1] prever estado
```

```
[2] testar métodos
```

```
[3] montar turnos
```

```
[4] vencer a batalha
```

Missão de hoje



REGRA DA AULA

Nenhum while antes dos métodos funcionarem manualmente.

Executar pouco código, observar o estado e explicar o resultado.

Aquecimento: leia a classe

PYTHON

```
class Personagem:
    def __init__(self, nome, vida, ataque):
        self.nome = nome
        self.vida = vida
        self.ataque = ataque

    def mostrar_status(self):
        print(self.nome, "-", self.vida, "de vida")

thoric = Personagem("Thoric", 100, 15)
thoric.mostrar_status()
```

IDENTIFIQUE

- 3 parâmetros de entrada
- 3 atributos de instância
- 1 método de leitura
- 1 objeto criado
- a saída do terminal

SAÍDA

```
> Thoric - 100 de vida
```

Preveja o estado após cada linha

SEM EXECUTAR AINDA

```
thoric = Personagem("Thoric", 100, 15)  
goblin = Personagem("Goblin", 45, 8)
```

```
thoric.receber_dano(20)  
goblin.receber_dano(12)  
thoric.receber_dano(200)
```

RASTREAMENTO

início	100	45
--------	-----	----

Thoric -20	80	45
------------	----	----

Goblin -12	80	33
------------	----	----

Thoric -200	0	33
-------------	---	----

Resposta: a vida nunca fica abaixo de zero.

Complete o construtor do RPG

COMPLETE

PYTHON

```
class Personagem:
    def __init__(self, nome, vida, ataque,
                  defesa, pocoes):
        _____ = nome
        _____ = vida
        _____ = vida
        _____ = ataque
        _____ = defesa
        _____ = pocoes
        _____ = False
```

SOLUÇÃO

PYTHON

```
self.nome = nome
self.vida = vida
self.vida_maxima = vida
self.ataque = ataque
self.defesa = defesa
self.pocoes = pocoes
self.defendendo = False
```

vida_maxima guarda o limite da cura.

Construa e teste receber_dano()

PYTHON

```
def receber_dano(self, quantidade):  
    dano_real = quantidade - self.defesa  
  
    if dano_real < 0:  
        dano_real = 0  
  
    self.vida -= dano_real  
  
    if self.vida < 0:  
        self.vida = 0
```

3 TESTES MANUAIS

```
thoric = Personagem(  
    "Thoric", 100, 15, 5, 2)  
  
thoric.receber_dano(20) # vida 85  
thoric.receber_dano(3)  # vida 85  
thoric.receber_dano(200) # vida 0
```

A regra pertence à classe:

todo personagem protege
o próprio estado.

Interação: atacar(alvo)

PYTHON

```
def atacar(self, alvo):  
    print(self.nome, "ataca", alvo.nome)  
    alvo.receber_dano(self.ataque)
```

```
thoric.atacar(goblin)
```

`self = thoric`

fonte de self.ataque

`alvo = goblin`

executa receber_dano()

`Thoric.ataque` `>` `Goblin.receber_dano()`

Leia da esquerda para a direita: quem age, qual método, quem recebe.

Defender vale para o próximo ataque

PYTHON

```
def defender(self):
    self.defendendo = True
    print(self.nome, "preparou a defesa")

def receber_dano(self, quantidade):
    dano_real = quantidade - self.defesa

    if self.defendendo:
        dano_real = dano_real // 2
        self.defendendo = False

    if dano_real < 0:
        dano_real = 0

    self.vida = max(0, self.vida - dano_real)
```

MUDANÇA DE ESTADO

início

False

defender()

True

receber dano

False

ataque seguinte

normal

Poção: recurso limitado e vida máxima

PYTHON

```
def usar_pocao(self):
    if self.pocoes == 0:
        print("Nenhuma poção restante")
    elif self.vida == self.vida_maxima:
        print("A vida já está cheia")
    else:
        cura = 25
        self.vida += cura

        if self.vida > self.vida_maxima:
            self.vida = self.vida_maxima

        self.pocoes -= 1
        print(self.nome, "usou uma poção")
```

TESTE OS LIMITES

- vida 60, poções 2 > vida 85, poções 1
- vida 90 > vida 100, nunca 115
- vida cheia > não gasta poção
- zero poções > não altera a vida

Dano variável e chance de acerto

PYTHON

```
import random

def atacar(self, alvo):
    rolagem = random.randint(1, 20)

    if rolagem < 5:
        print(self.nome, "errou o ataque")
    else:
        dano = random.randint(
            self.ataque - 3,
            self.ataque + 3
        )
        alvo.receber_dano(dano)
```

d20: 1 até 20

1, 2, 3, 4 = erro

5 até 20 = acerto

ORDEM IMPORTANTE

1. rolar acerto
2. calcular dano se acertar

No erro, receber_dano() não executa.

O menu chama métodos já testados

PYTHON

```
def turno_do_jogador(jogador, inimigo):  
    print("1 - Atacar")  
    print("2 - Defender")  
    print("3 - Usar poção")  
  
    escolha = input("Escolha: ")  
  
    if escolha == "1":  
        jogador.atacar(inimigo)  
    elif escolha == "2":  
        jogador.defender()  
    elif escolha == "3":  
        jogador.usar_pocao()  
    else:  
        print("Opção inválida")
```

input() retorna string

"1" == "1" True

"1" == 1 False

SEM DUPLICAR REGRAS

menu escolhe a ação

método executa a ação

Monte o laço da batalha na ordem certa

PYTHON

```
while jogador.vida > 0 and inimigo.vida > 0:
    jogador.mostrar_status()
    inimigo.mostrar_status()

    turno_do_jogador(jogador, inimigo)

    if inimigo.vida > 0:
        inimigo.atacar(jogador)

if jogador.vida > 0:
    print("Você venceu!")
else:
    print("Você foi derrotado.")
```

FLUXO DE UM TURNO

1. conferir duas vidas
2. mostrar status
3. jogador escolhe
4. conferir inimigo
5. contra-atacar se vivo

Depuração: cinco erros do nosso RPG

ENCONTRE ANTES DE REVELAR

```
class Personagem:
    def __init__(nome, vida):          # 1
        nome = nome                   # 2
        self.vida = vida

escolha = input("1 - Atacar: ")
if escolha == 1:                      # 3
    jogador.atacar(inimigo)

while jogador.vida > 0 or inimigo.vida > 0: # 4
    turno_do_jogador(jogador, inimigo)
    inimigo.atacar(jogador)           # 5
```

CORREÇÕES

- Adicionar self no método
- Usar self.nome
- Comparar com "1"
- Trocar or por and
- Conferir vida do inimigo

Erro mostra o sintoma.

Estado mostra a causa.

Vários inimigos, o mesmo jogador

PYTHON

```
inimigos = [  
    Personagem("Goblin", 40, 8, 2, 0),  
    Personagem("Orc", 70, 12, 4, 0)  
]  
  
for inimigo in inimigos:  
    print("Novo inimigo:", inimigo.nome)  
  
    while jogador.vida > 0 and inimigo.vida > 0:  
        turno_do_jogador(jogador, inimigo)  
  
        if inimigo.vida > 0:  
            inimigo.atacar(jogador)  
  
    if jogador.vida == 0:  
        break
```

for externo

troca o inimigo

while interno

resolve uma batalha

vida e poções continuam

no mesmo jogador

Mapa do RPG completo



main cria objetos > combate repete > turno escolhe > métodos alteram estado

Desafios finais: duas evoluções do mesmo RPG

MÉDIO

Ataque crítico com d20

- 1 a 4: erra
- 5 a 18: dano normal
- 19 ou 20: dano dobrado
- mostrar rolagem e dano no terminal
- testar sem duplicar receber_dano()

MÉDIO +

Inimigo curandeiro

- Orc Xamã começa com 1 poção
- sorteia atacar ou usar poção
- não cura com vida máxima
- não age depois de derrotado
- estado continua entre os turnos

ENTREGA

3 testes normais + 2 casos de borda + explicação do fluxo escolhido

Folha de consulta rápida

__INIT__

```
def __init__(self, vida):  
    self.vida = vida
```

SELF + ALVO

```
jogador.atacar(inimigo)  
self=jogador | alvo=inimigo
```

INPUT

```
escolha = input(...)  
if escolha == "1":
```

RANDOM

```
random.randint(1, 20)  
limites incluídos
```

WHILE

```
while jogador.vida > 0 and  
    inimigo.vida > 0:
```

DEPURAR

```
ler erro > conferir tipo >  
conferir objeto > conferir estado
```

Teste seguro: criar > chamar > mostrar estado > provocar um caso de erro.